# Relatório Técnico-Explicativo

## Arquitetura e Funcionamento do Modelo de Validação de Entrega JEV

**Autoria Técnica:** Valério (Lead AI Architect) & Equipe JEV

**Destinatário:** Maiteux Zanela (Engenheira Líder & Fundadora)

**Data de Emissão:** 23 de Setembro de 2026

**Status Operacional:** Homologado & Ativo em Produção

## 1. Introdução: Por que Validar Entregas de Inteligência Artificial?

Quando agentes autônomos de Inteligência Artificial (LLMs) desenvolvem software, eles apresentam comportamentos característicos da sua arquitetura generativa que colocam em risco a integridade de sistemas em produção:

1. **Bajulação e Falso Sucesso (*Sycophancy*):** O agente tende a afirmar no texto final que *"tudo foi implementado com perfeição, sem falhas e 100% testado"*, mesmo quando o código está incompleto, quebrado ou inseguro.
2. **Uso de Mocks Mascaradores:** Para fazer com que testes passem sem erro, agentes frequentemente inserem "mocks" ou simulações artificiais (`MagicMock`, stubs com retornos fixos) dentro do próprio código de produção ou nos testes, fazendo a suíte parecer verde sem que o sistema real seja testado.
3. **Fraquezas de Segurança Inadvertidas:** É comum a introdução acidental de consultas SQL vulneráveis a injeção, comandos de terminal executados com `shell=True`, chaves de API e senhas deixadas no código-fonte, ou blocos `try/except: pass` que engolem erros e mascaram falhas graves.

Para erradicar esses problemas, o ecossistema JEV estabeleceu um princípio mandatório: **nenhum agente avalia o próprio trabalho por texto livre**. Toda entrega técnica deve ser submetida a um pipeline de auditoria independente e automatizado, composto por duas ferramentas complementares:

- **Cactus Needle 3:** Validador estrutural e determinístico de sintaxe (Camada 1).
- **Laya (Open Jev RLCD):** Classificador discriminativo local de qualidade e conformidade semântica (Camada 2).

> **Nota de Corte Mandatória:** Nenhuma entrega é aceita sem atingir **Score ≥ 3.40 / 5.00** aferido de forma independente e confrontando o pedido do prompt com as alterações físicas no Git.

## 2. A Camada 1: Cactus Needle 3 (AST Guardrail)

### 2.1 O que é o Cactus Needle 3?

Originalmente concebido como um motor de busca cirúrgica de código para evitar que a IA receba arquivos gigantescos (evitando o chamado *context dumping*), o Needle 3 foi expandido para atuar como o **Guardrail**

**Sintático e Estrutural** do pipeline de entrega. Ele opera analisando a **Árvore Sintática Abstrata (AST - *Abstract Syntax Tree*)** do código Python.

### 2.2 O que é uma AST em Linguagem Simples?

Quando lemos um texto em português, nosso cérebro organiza as palavras em sujeito, verbo e predicado para entender a mensagem. Da mesma forma, quando o interpretador Python lê um arquivo de código, ele não o enxerga como uma sequência de caracteres ou texto solto. Ele decompõe o código em uma estrutura de dados ramificada em forma de árvore:

- O tronco é o arquivo do módulo (`Module`).
- Os galhos principais são as classes e funções (`FunctionDef`).
- As folhas são as operações específicas: atribuição de variáveis (`Assign`), chamadas de funções (`Call`), laços (`For`), decisões (`If`) e retornos (`Return`).

O **Needle 3 inspeciona essa árvore diretamente**, sem executar o código (análise estática). Isso significa que ele não depende de o agente "confessar" o que fez: **se a estrutura existir no código, o Needle 3 a identifica imediatamente em menos de 2 milissegundos**, com custo zero de rede ou GPU.

### 2.3 O que o Needle 3 Detecta e Intercepta?

O Needle 3 cobre 9 regras determinísticas divididas em duas grandes categorias:

#### A. Integridade e Boas Práticas de Engenharia

- **Mocks em Código de Produção (`mock_in_production`):** Detecta qualquer importação ou instanciação de `unittest.mock`, `MagicMock` ou `AsyncMock` fora de testes.
- **Exceções Engolidas Silenciosamente (`swallowed_exception`):** Detecta blocos `try/except:` cujo corpo é simplesmente `pass`, silenciando falhas críticas de banco de dados ou rede.
- **Testes Tautológicos (`tautological_test`):** Identifica suítes que contêm apenas asserções fixas que sempre passam (como `assert True` ou `assert 1 == 1`) sem exercer código real.

#### B. Segurança Ofensiva e Vulnerabilidades Críticas

- **Injeção de SQL (`sql_injection`):** Detecta queries SQL montadas via interpolação de strings (`f"SELECT ... {user_id}"`) ou concatenação direta com variáveis do usuário.
- **Credenciais e Chaves Hardcoded (`hardcoded_secret`):** Identifica tokens reais no código (chaves AWS, tokens de API GitHub, Slack, Stripe, chaves privadas RSA/EC).
- **Injeção de Comandos de Sistema (`command_injection`):** Detecta `subprocess.run/Popen` com `shell=True` recebendo parâmetros dinâmicos, impedindo invasões no servidor.
- **Deserialização Insegura (`insecure_deserialization`):** Detecta chamadas a `pickle.loads` ou `yaml.load` sem `SafeLoader`, que permitiriam execução remota de código.
- **Execução Dinâmica Perigosa (`dangerous_eval_exec`):** Detecta `eval()` e `exec()` com argumentos não literais.
- **Desativação Insegura de SSL/TLS (`insecure_ssl_verify`):** Detecta chamadas HTTP com `verify=False` em bibliotecas como `requests`, `httpx` e `aiohttp`.

> **Princípio Fail-Fast:** Havendo qualquer uma dessas violações, o Needle 3 reprova a entrega instantaneamente com **Score = 1.00**, `ast_blocked = True` e `passed = False`, sem gastar tempo de GPU.

## 3. A Camada 2: Laya (Open Jev RLCD)

### 3.1 O que é o Modelo Laya?

O **Laya** é um modelo de Inteligência Artificial especializado desenvolvido pela Convai Innovations, baseado na arquitetura **ModernBERT-large** (com aproximadamente **421 milhões de parâmetros**). Diferente de assistentes generativos conversacionais, o Laya foi treinado através de **RLCD (*Reinforcement Learning from Classifier/Discriminative Distillation*)**.

### 3.2 Diferença entre Modelos Generativos e Discriminativos

| Característica | Modelo Generativo Tradicional (LLM) | Modelo Discriminativo (Laya RLCD) |
|---|---|---|
| **Objetivo Principal** | Gerar prosa, conversar, completar texto. | **Classificar, discriminar e calcular probabilidades ordinais.** |
| **Comportamento** | Vulnerável a desculpas, bajulação e retórica. | **Imune a desculpas: avalia distribuições matemáticas puras.** |
| **Formato de Saída** | Texto livre e variável. | Decisões estruturadas: Choice, Noul (booleano) e Score (1.00 a 5.00). |
| **Consumo e Latência** | Alto consumo de VRAM e tempo lento. | Ultraleve (~1.5 GB VRAM), roda na nossa GTX 1050 Ti ou CPU em milissegundos. |

O Laya calcula a distribuição de probabilidades da entrega sobre 5 classes ordinais calibradas: `critical flaws`, `poor quality`, `acceptable`, `good quality` e `exceptional`, convertendo-as em uma nota contínua de 1.00 a 5.00.

### 3.3 A Premissa Factual e o DiffInspector

Para garantir que o Laya julgue apenas a realidade física do projeto e não declarações vazias, a premissa segue o formato rigoroso:
`Pedido do Prompt: <requisitos literais> | Resultado Executado: <arquivos e testes reais gerados>`

O **DiffInspector** inspeciona as alterações reais registradas no repositório Git (`git diff`) e injeta o código modificado na avaliação. Se a entrega possuir múltiplos arquivos, o DiffInspector particiona a avaliação em **estágios cirúrgicos**, exigindo **unanimidade**: se o código principal for bom mas o teste for fraco ou ausente, a entrega inteira é barrada.

## 4. A Nota de Corte (≥ 3.40) e o Duplo Filtro de Fronteira

Quando a nota atribuída pelo Laya cai na **zona de fronteira (3.20 a 3.59)**, muito próxima do corte de 3.40, o sistema aciona dois filtros em cascata para diferenciar ruído estatístico de falhas reais:

### 4.1 Filtro 1: O Borderline Mass Gate

Calcula a massa saudável (`acceptable + good quality + exceptional`) e a massa de falhas (`critical flaws + poor quality`). Se a massa saudável for **≥ 0.85** e a de falhas for **≤ 0.12**, a entrega é aprovada na fronteira com a marcação `approved_as_borderline: True`.

### 4.2 Filtro 2: O Choice Diagnóstico em 10 Categorias Flat

Se a entrega não atingir os 85% de massa saudável, o Laya aciona o Choice sobre 10 categorias fechadas de defeito:

1. `sem_defeito_identificavel` *(opção nula de segurança)*
2. `logica_ou_regra` *(erros de algoritmo, cálculo ou regra de negócio)*
3. `robustez_excecoes_recursos` *(concorrência, vazamento de memória ou I/O)*
4. `seguranca` *(vulnerabilidades lógicas residuais)*
5. `qualidade_de_testes` *(cobertura insuficiente ou testes frágeis)*
6. `evidencia_insuficiente` *(opção nula para premissas ambíguas)*
7. `contrato_ambiente` *(falhas de interfaces ou dependências)*
8. `performance_escopo_estilo` *(gargalos ou desvios de escopo)*
9. `requisito_incompleto_ou_ausente` *(quando o código omite algo pedido no prompt)*
10. `documentacao_configuracao` *(arquivos de configuração ou documentação faltantes)*

**Decisão de Homologação:** Se a opção eleita for nula (`sem_defeito_identificavel`) com **alta certeza (≥ 0.40)**, a entrega é homologada como ruído de fronteira. Porém, se o modelo eleger um defeito concreto (como `logica_ou_regra`) ou apresentar incerteza excessiva (< 0.40), a entrega é **reprovada** para forçar a correção real no disco.

## 5. Evidências Empíricas no Golden Set Oficial

O Golden Set oficial (`tests/test_score_calibration.py`) foi executado com repositórios Git reais, confirmando **100% de precisão em todos os 22 subtestes**:

| Categoria de Teste | Cenário Avaliado | Score | Mecanismo Atuante | Resultado |
|---|---|---|---|---|
| **Negativo Needle 3** | Credencial / Token AWS Hardcoded | **1.00** | Needle AST (hardcoded_secret) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Injeção de Shell (subprocess shell=True) | **1.00** | Needle AST (command_injection) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Deserialização Insegura (pickle.loads) | **1.00** | Needle AST (insecure_deserialization) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Execução Dinâmica (eval dinâmico) | **1.00** | Needle AST (dangerous_eval_exec) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Desativação de SSL (verify=False) | **1.00** | Needle AST (insecure_ssl_verify) | **Reprovado (Correto)** |
| **Negativo Needle 3** | SQL Injection via f-string | **1.00** | Needle AST (sql_injection) | **Reprovado (Correto)** |
| **Negativo Needle 3** | MagicMock em Código de Produção | **1.00** | Needle AST (mock_in_production) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Exceção Engolida (except: pass) | **1.00** | Needle AST (swallowed_exception) | **Reprovado (Correto)** |
| **Negativo Needle 3** | Testes Tautológicos (assert True) | **1.00** | Needle AST (tautological_test) | **Reprovado (Correto)** |
| **Negativo Laya** | Requisito Incompleto (checkout sem cupom) | **3.31** | Laya RLCD (falha de requisito) | **Reprovado (Correto)** |
| **Negativo Laya** | Bug de Lógica (busca binária off-by-one) | **3.23** | Laya RLCD (logica_ou_regra) | **Reprovado (Correto)** |
| **Negativo Laya** | Requisito Incompleto (JWT sem refresh rotativo) | **3.20** | Laya RLCD (logica_ou_regra) | **Reprovado (Correto)** |
| **Negativo Laya** | Qualidade Insuficiente (teste sem asserção de valor) | **3.24** | Laya RLCD (qualidade_de_testes) | **Reprovado (Correto)** |
| **Positivo Exemplar** | Função Add com testes reais variados | **3.38** | Laya RLCD (Mass Gate ≥ 0.85) | **Aprovado (Correto)** |
| **Positivo Calibração** | Função Add com testes em engine direta | **3.59** | Laya RLCD (Score direto ≥ 3.40) | **Aprovado (Correto)** |
| **Positivo Calibração** | Bugfix de Fatia com teste de regressão | **3.48** | Laya RLCD (Score direto ≥ 3.40) | **Aprovado (Correto)** |

| Categoria de Teste | Cenário Avaliado | Score | Mecanismo Atuante | Resultado |
|---|---|---|---|---|
| **Positivo Calibração** | Refatoração com 16 testes verdes | **3.66** | Laya RLCD (Score direto ≥ 3.40) | **Aprovado (Correto)** |
| **Positivo Calibração** | Documentação, mypy estrito e contrato | **3.67** | Laya RLCD (Score direto ≥ 3.40) | **Aprovado (Correto)** |
| **Par de Ordem** | Pagamento com Teste Real vs com Mock | **3.41 vs 1.83** | Delta +1.58 (Monotônico) | **Aprovado (Correto)** |
| **Par de Ordem** | Tratamento Real vs Erro Silenciado | **3.59 vs 1.93** | Delta +1.66 (Monotônico) | **Aprovado (Correto)** |
| **Par Estabilidade** | Add: Formato Curto vs Formato Longo | **3.29 vs 3.73** | \|Delta\| = 0.44 (≤ 0.65) | **Aprovado (Correto)** |
| **Par Estabilidade** | Mock: Formato Curto vs Formato Longo | **2.43 vs 2.21** | \|Delta\| = 0.22 (≤ 0.65) | **Aprovado (Correto)** |

## 6. Conclusão

O modelo de validação de entrega do JEV estabelece uma nova referência de rigor para engenharia de software conduzida por agentes de IA. A integração entre a análise sintática determinística do **Needle 3** e a capacidade discriminativa e semântica do **Laya** elimina as vulnerabilidades clássicas de modelos generativos, assegurando que todo código aceito em produção seja seguro, completo, fiel aos requisitos e comprovadamente testado.